ChainNative Docs Operations Manual

Git Sync + Claude Code

Document Production Flow / Operations Rules / FAQ & Docs Design

PART 1: ChainNative Docs Operations Manual (Git Sync + Claude Code)

0. How It Works

A. Mechanism

- When a GitHub repository is connected to a GitBook space, any git push to the branch is automatically reflected in GitBook within a few seconds.
- Because the sync is bidirectional, merging a change request on GitBook also generates a commit in GitHub.

> ** Actual operation flow with Claude Code:*
>
> - Give Claude Code a topic instruction.
> - Claude generates a Markdown file (including heading structure / tables / images).
> - SUMMARY.md is automatically updated.
> - git push
> - Changes are automatically reflected in GitBook.

B. Feature Overview

1. Article Creation (with minimal instructions)

- Simply passing an instruction sheet to Claude Code generates an article from scratch with an H1/H2/H3 heading structure.
- Loading the official skill.md provided by GitBook into Claude Code's context enables accurate output in GitBook-specific formatting.

2. Tables

- Fully supported via GFM pipe notation (| col1 | col2 |).
- Structured content such as parameter tables and comparison tables is the type of content Claude handles best.

3. Images

- Place image files in the .gitbook/assets/ folder and embed them using standard Markdown notation (![alt](path)).
- Embedding images from external URLs is also supported.
- Mermaid notation can be used for architecture diagrams.

C. Important Configuration Notes

> ** 1. Pages not listed in SUMMARY.md will not appear in GitBook. Always include 'also update SUMMARY.md' in instructions to Claude Code.*

> ** 2. When Git Sync is enabled, direct editing in the GitBook UI is locked. All changes must go through the change request flow. Establish the operations workflow in advance.*

1. What You Can Do with This Setup

1. Using Git Sync + Claude Code, you can edit, add, and update merchant-facing GitBook documentation simply by instructing Claude Code.
2. Once Claude uploads (pushes) the edited files to GitHub, they are automatically reflected in the published GitBook documentation within a few seconds.

Main Operations Available

- Edit, rewrite, or fix typos in existing pages.
- Add new pages (setup guides, FAQ pages, feature descriptions, etc.).
- Delete pages or change their order.
- Translate from Japanese to English or from English to Japanese.
- Unify phrasing and writing style (rewriting into clear, merchant-friendly language, etc.).
- Batch edits across multiple pages.

> ** Design changes (theme colors, font settings, etc.) and image uploads cannot be done via Claude Code. These must be done directly in the GitBook admin panel.*

2. One-Time Setup (Required per Environment)

A. Download the Repository from GitHub to Your PC (Clone)

"Clone" means copying the repository (folder) saved on GitHub to your local PC. Perform this once per PC used for work.

Step 1: Open PowerShell

- Right-click the Start button and select "Windows PowerShell".

Step 2: Navigate to the Documents folder

Step 3: Clone the repository

```
git clone https://github.com/ChainNative/chainnative-docs
```

- Once complete, a folder named "chainnative-docs" will be created inside your Documents folder.

3. Standard Workflow (Steps to Follow Each Time)

A. Launch Claude Code

Step 1: Open PowerShell

- Right-click the Start button and select "Windows PowerShell".

Step 2: Navigate to the repository folder

- Always navigate to the same folder before launching Claude Code.
- Launching from a different folder will prevent Claude from recognizing the GitBook files.

```
cd C:\Users\[username]\Documents\chainnative-docs
```

Step 3: Fetch the latest files from GitHub

```
git pull origin main
```

> ** Always run this step when multiple people are editing, or when edits have been made directly on the GitBook side. This ensures you start from the latest state.*

Step 4: Launch Claude Code

```
claude
```

- If a trust confirmation prompt appears after launch, select "1. Yes, I trust this folder" and press Enter.

B. Giving Edit Instructions to Claude / Common Operation Patterns

3. Add a new page — Claude automatically handles the SUMMARY.md update at the same time.
4. Edit an existing page
5. Rewrite a page
6. Translate a page
7. Batch edit across multiple pages
8. Check the current structure
9. Delete a page

C. Review the Changes

10. When Claude finishes editing, a list of changed files and their contents is displayed.
    - Review the content. If there are no issues, proceed to the next step.
    - If edits are needed, give additional instructions to Claude Code directly.

D. Upload to GitHub and Reflect in GitBook (PR Flow)

11. Do not push directly to the main branch.
    - Create a branch named docs/article-name, push the changes, then create a Pull Request on GitHub.
    - Include a summary of the changes in the PR description.
    - Merging is performed by the PM after approval.

Run the following commands in order:

Step 1: Create a branch

```
git checkout -b docs/article-name
```

Step 2: Stage all changed files

```
git add .
```

Step 3: Save with a commit message

```
git commit -m "Brief description of the changes"
```

> ** Example commit messages:*
>
> - git commit -m "Add wallet setup guide"
> - git commit -m "Rewrite Walkthrough page for merchants"
> - git commit -m "Translate Wallet FAQ into English"

Commit messages are used to review change history later. Write something that makes it immediately clear what was changed.

Step 4: Push to GitHub

```
git push origin docs/article-name
```

12. After the PM merges the PR to main, the published GitBook documentation is automatically updated within a few seconds.

4. Troubleshooting

A. Error when running git push

If changes have been made from another device or directly on the GitBook side, you may not be able to push directly. Fetch the latest state with the following commands and retry.

```
git pull origin main
git push origin main
```

B. Claude Code won't launch

13. Restart PowerShell and try again.
14. If it still fails, check the following:

```
claude --version
```

    - If a version number is displayed, Claude Code is installed correctly.
    - If nothing is displayed, Claude Code needs to be reinstalled.

C. chainnative-docs folder not found

15. The initial clone may not have been completed. Follow the steps in Section 2 to perform the clone.

PART 2: ChainNative Document Production Flow / Operations Rules

1. Fixed Rules (Applies to All Articles)

A. Output Format

- Format: Markdown for GitBook
- Language: English / Japanese
- Headings: Use H1 / H2 / H3 structure

B. Items Prohibited from Publication

16. Internal system names / internal project document names (e.g., internal document names such as "ChainNative Support Design")
17. Internal URLs / admin panel URLs
18. Actual values of API keys / secrets / signatures
19. Production environment IP addresses / server configuration details
20. Unpublished features / roadmap information

> ** If any of the above is found in a draft, Claude flags it as a candidate item → the PM makes the final judgment.*

2. Category / Theme List Management (via GitHub Issues)

A. FAQ Research / List Compilation

21. Add items on an ongoing basis based on merchant inquiry logs (dev/docs team).
    - Tag new inquiries that should be made into FAQs.
    - Review the list of tagged tickets from L1 and identify FAQ candidates.

22. Technical team (dev/infra) picks up items.
    - Items that need to be produced, such as setup guides.
    - Share in advance if there are specific content requirements.

23. Research by the Docs team.
    - Identify missing articles.
    - The PM may provide candidate suggestions.

B. Log and Manage via GitHub Issues with Labels (Docs Team)

24. Compile the category / theme list first. Do not create articles directly without listing first.
25. Check whether any existing articles are similar in content. If close, add to / merge with the existing article and keep the URL (avoid URL changes wherever possible).

3. Heading Candidate Request (One Sheet per Article)

A. Instruct Claude to Generate Content and Heading Candidates (Docs Team)

26. Confirm the category / theme. Select from GitHub Issues. The Docs team submits an "addition/approval request" → the PM checks and approves.
27. After approval, create the instruction sheet:
    - Issue number
    - Category
    - Theme
    - Reference information / supplementary notes (additional context to pass, such as existing related page URLs or specification information to reference)

28. Create Candidate List A: Claude generates candidates from the instruction sheet.
    - Candidate items to be covered.
    - Heading structure candidates based on those items (H1 / H2 / H3...).

B. Final Confirmation / Approval (PM)

29. Review and revise Candidate List A.

30. After final confirmation, add a confirmation signature and date as a record of approval.

4. Draft Generation Request (One Sheet per Article)

A. Request Claude to Generate a Draft (Docs Team)

Template:

> ** Please create an article for merchants in Markdown for GitBook under the following conditions:*
>
> - Content to include: [Paste the finalized instruction sheet as-is]
> - After generating the draft, list any items that may be technically inaccurate or that may not be suitable for publication.
> - Output language: English
> - Format: Markdown for GitBook

5. Formatting / Screenshots / SUMMARY.md Update / PR Creation (Docs Team)

A. Formatting / Checking Draft Revisions

31. Use Claude as a sounding board to finalize the article.
    - Compile any questions or items requiring confirmation for the PM and ask them all at once.
    - Capture screenshots, insert video links, create and insert diagrams (confirm with the technical team for any assets needed).
    - Add tables and code examples as needed.
    - Unify terminology and tone (adjust wording to be clear and unambiguous from a merchant perspective).

B. SUMMARY.md Management Rules

32. SUMMARY.md is the file that defines the entire page structure of GitBook. Any Markdown file not listed here will not appear in GitBook. Always update SUMMARY.md when adding a new page.
33. PM approval is required for: adding / removing / reordering sections, and approving changes to existing URLs.

C. Screenshot / Image Management

34. Place all image files in the .gitbook/assets/ folder.
35. Use the following naming format: feature-name_description_date.png — e.g., webhook_signature_verification_20260401.png

36. Use standard Markdown notation for insertion: ![description](.gitbook/assets/filename.png)
37. Check that no prohibited items (API keys, secrets, internal URLs, etc.) are visible in screenshots.

D. Create a PR

38. After formatting is complete, create a Pull Request. Do not push directly to main — create a branch named docs/article-name. Include any notes in the PR description.

6. Review / Publication Decision (PM)

39. Review is conducted by Claude. Have Claude review the diff and list any items that may be technically inaccurate or may violate publication rules.
40. Make the final publication / non-publication decision. If revisions are needed, send back to the Docs team.
41. Approve and publish (PM always retains final merge approval and publication decision). Merge work is carried out using Claude. If there are no issues, merge to main → GitBook is automatically updated within a few seconds.

7. Confirm Reflection in GitBook (Docs Team)

42. Confirm that the new upload or change is correctly reflected.
43. If not reflected in GitBook:
    - Clear the browser cache (Ctrl + Shift + R).
    - Access app.gitbook.com and check the Git Sync status of the target space.
    - In the GitBook admin panel, confirm that the Sync status shows "Active".

8. Update / Deprecation Flow (Existing Page Maintenance / Post-Scale)

A. Enable Reactions / Feedback Features (Premium Plan or Higher)

- Turn on GitBook's Reactions (good/bad buttons) and comment features.
- Ensure feedback can be collected on all published pages.

B. Regular Monitoring / Update Triggers

44. Check inquiry logs weekly / monthly. Quickly scan for Docs with low satisfaction. Articles where "reading this page didn't resolve the issue" keeps coming up are candidates.
45. Identify low-rated pages (rough review is fine). Low-reaction (bad button) pages above a threshold; check pages with the worst access numbers / highest bad ratings.

46. UI changes / specification changes / version upgrades — requests from the technical team.

C. Check and Understand the Diff / Problem Areas

- What is outdated: screenshots? procedures? cautions?
- Why is it not resolving the issue? etc.

D. Maintenance Flow

47. Identify issues / create revision proposals. Request research from Claude. Do not accept research results at face value — conduct multi-angle research.
48. Share revision proposals with the PM. After approval, request a rewrite. Recreate any images or tables that need to be remade. Create a PR.
49. Review → re-publication decision. Same as the standard flow.

E. Deprecation Decision

- Features that are no longer in use / old versions.
- Move to an "Archive" section or unpublish.

PART 3: FAQ / Docs Design

Content differs between sections, so they should be separated at the entry point and structurally.

> ** Reference service: https://help.glpi-project.org/*

GitBook Structure

A. Top-Level Structure within One Space

ChainNative Docs

- ├─ Before You Start (pre-login / considering adoption)
- └─ For Merchants (post-login / in operation)

Header categories:

- Non-merchant side: sales/marketing-oriented questions (service overview, pricing, risk, adoption requirements, etc.)
- Merchant side: practical / technical / troubleshooting (setup, operations, technical FAQs, incident response, etc.)

B. GitBook Top (Welcome) Structure

Use a category card layout to separate entry points.

1. Before You Start ChainNative

- Target: Non-merchants (considering adoption, pre-login users)
- Content: FAQs and light guides on service overview / pricing / risk / adoption conditions.

2. ChainNative for Merchants

- Target: Merchants already under contract (post-login users)
- Content: Setup procedures, operations guides, troubleshooting, technical FAQs.

3. Developers / API (to be implemented separately post-scale)

- Target: Integration developers
- Content: API specifications, Webhooks, samples, etc.

Before You Start ChainNative

Purpose: Help users decide whether to adopt ChainNative / reduce misconceptions.

- Target: Non-merchants (considering adoption, pre-login users)
- Content: FAQs and light guides on service overview / pricing / risk / adoption conditions.
- Message: Is this service right for me? What are the costs / risks / adoption hurdles?

Category 1: What is ChainNative?

- What does ChainNative do? (Service overview / payment gateway mechanism)
- How is it different from other payment services? (No chargebacks / freeze risk / non-custodial)
- Which cryptocurrencies are supported?
- Which countries / regions can use it?

Category 2: Who is ChainNative For?

- What businesses / use cases is ChainNative suited for? (High-chargeback industries / businesses struggling with payment freezes, etc.)
- What businesses / use cases is it NOT suited for? (Illegal purposes / regulatory No-Go)
- Can it be used alongside existing payment methods (card payments, etc.)?

Category 3: Pricing & Plans

- Plan comparison (differences between plans / onramp availability / support level)
- Fee structure (subscription fee / transaction fee)
- Handling of fees for refunds / cancellations
- Can plans be changed or cancelled?

Category 4: Requirements & Compatibility

- Supported platforms / environments (own website / payment links / Telegram integration, etc.)
- What is needed to get started (BTC wallet / target countries / currencies / approach to regulations)
- Technical prerequisites for adoption (is an engineer required?)

Category 5: Risk & Compliance

- What is non-custodial? (Merchant holds key management responsibility / precautions under non-custodial premise)
- How are funds managed?
- What does "no chargebacks" mean? Benefits and caveats.
- Responsibility boundaries between ChainNative and the merchant for KYC/AML
- Approach to regulatory risk (which purposes / countries are out of scope?)
- How is cryptocurrency price volatility risk handled?

Category 6: Setup Overview

- Overall flow from adoption to Go-Live (application → KYC → review → setup → test payment)
- List of required documents / information

ChainNative for Merchants

Purpose: Resolve operational blockers as quickly as possible / reduce support load.

- Target: Merchants already under contract (post-login users)
- Content: Setup procedures, operations guides, troubleshooting, technical FAQs.
- Message: How to set up and run day-to-day operations. Knowing at a glance where to look when something goes wrong.

Category 1: Getting Started

- Account creation / initial setup procedure
- Wallet connection guide (hot wallet / seed phrase precautions)
- Steps to receive your first payment
- Basic dashboard operations

- Enabling / requiring Two-Factor Authentication
- Email server setup
- Test payment verification

Category 2: Payments & Invoices

- Creating and managing invoices (sharing links via email / chat)
- How to use Payment Requests (sharing links via email / chat)
- Implementing ChainNative Checkout
- Setting up the CN Pay Button
- Payment flow overview
- How to check payment status
- Transaction status definitions (unconfirmed / confirmed / failed, etc.)
- Refund / reimbursement procedures

Category 3: Webhooks

- Webhook setup procedure
- Signature verification implementation guide
- Timestamp verification / replay attack countermeasures
- Idempotency implementation guide

Category 4: Reporting & Reconciliation

- Overview of the Reporting screen and each report type (Payments / Refunds / Sales, etc.)
- How to aggregate sales and export to CSV
- How to read fee calculations and sales reports (by period / currency)
- Handling of unconfirmed transactions (when to count as revenue)

Category 5: Operations & Best Practices

- Staff permission design
- Daily / weekly checklist items (unprocessed invoices / errors / Webhook failures, etc.)

Category 6: Security & Compliance (for Merchants)

- IP restrictions / permission management
- Handling suspicious deposits / addresses
- Log retention period and audit approach

Category 7: Troubleshooting

- Checklist for payment not reflected / TXID not confirmed

- Steps to check when Webhooks are not arriving
- How to handle being unable to log in to the account
- Information merchants should prepare during an incident (TXID / log_reference_id / timestamp, etc.)

Category 8: Wallet Management

- How receiving and sending funds works, and the operational policy
- Preparing destination addresses
- How to check Transaction history
- Send operation (sending to an external wallet)

Category 9: Manage Billing

- What is a subscription? (Upgrade / downgrade procedure, information about the Trial, etc.)
- Billing cycle / payment method explanation
- Account cancellation (procedure flow, data retention period after cancellation, etc.)